

Introduction to Machine Learning — 시험 대비 총정리

UNLV, Mingon Kang 교수님 강의 (358p 전체 슬라이드 기반) — 리뷰세션 공지 반영판

이 노트는 한국어로 개념을 이해하고, 시험이 영어이므로 **모든 핵심 용어를 영→한 병기**했습니다.
각 챗터 끝에 "시험 포인트"를 넣어서 T/F 함정, 서술형에 나올 법한 질문을 표시했습니다.
★ 표시는 **강사가 리뷰세션에서 직접 강조한 출제 비중**을 나타냅니다 (★★★=최우선, ★★=중요, ★=1문제 정도).

시험 형식 (총 20문제, 전부 영어 / 계산기·노트 불가)

유형	문제 수	비고
True/False (참/거짓)	10문제	
Short Answer (단답형)	5문제	1~2문장으로 답변
Essay / Workout Problems (서술형·풀이형)	5문제	이 중 1문제는 Euclidean distance(유클리디안 거리) 계산 문제

출제 범위 우선순위 (강사 리뷰세션 강조 내용 기준)

우선순위	주제	출제 포인트
★★★ 최우선	Neural Networks & Backpropagation	비중 가장 큼. 단일 퍼셉트론의 한계와 MLP 필요성, Forward pass 단계, Backprop 원리, 입력/출력층 노드 수 설정 이유, 활성화 함수 선택
★★	KNN	알고리즘 작동방식·pseudocode, 적용 가능한 문제 유형 (약 2문제)
★★	SVM	계산보다 개념 위주 (약 2문제). Margin 최대화의 이유, Linear separation/판별자 개념
★★	Supervised vs Unsupervised Learning	차이점과 의미
★★	Data Evaluation & Preprocessing	Overfitting/Underfitting, Normalization, Optimization, Train-test split과 K-fold CV를 "하는 이유"
★★	Linear & Ridge Regression	최적 파라미터 찾는 방법, Multicollinearity(다중공선성) 문제, Ridge 공식과 Penalty term의 의미
★	Logistic Regression	약 1문제
—	Euclidean Distance 계산	Essay/Workout 5문제 중 반드시 1문제 출제 확정 → 아래 "계산 연습" 섹션 필수로 손으로 풀어볼 것

➤ **공부 순서 추천:** 이 우선순위표 순서대로 아래 본문을 읽는 것을 추천합니다. 특히 9번(신경망) 섹션은 이번 시험의 핵심이므로 가장 많은 시간을 투자하세요.

0. Introduction (AI vs Data Mining vs Machine Learning) & ★★ Supervised vs Unsupervised

- Machine Learning (머신러닝): Herbert Simon의 정의 — "경험(데이터)으로부터 성능(정확도)을 향상시키는 알고리즘 과정"
- AI (인공지능): 지능형 에이전트를 만드는 연구. 반드시 학습(learning)을 포함할 필요는 없음
- Data Mining (데이터 마이닝): 머신러닝/통계 기법을 빌려와 대량 데이터에서 지식을 발견(knowledge discovery)하는 것
- 세 개념은 겹치지만 동일하지 않음 — 이 차이가 T/F로 자주 나옴

Types of Learning (학습 유형)

영어	한국어	설명
Supervised learning	지도학습	라벨(정답)이 있는 데이터로 학습 → Classification(분류), Regression(회귀)
Unsupervised learning	비지도학습	라벨 없는 데이터에서 숨은 패턴 추론 → Clustering(군집화)
Semi-supervised learning	준지도학습	라벨/비라벨 데이터 혼합
Feature Selection/Reduction	특징 선택/축소	관련 있는 특징 부분집합 선택
Ensemble Learning	앙상블 학습	여러 모델 결합

★★ Supervised vs Unsupervised Learning — 차이점과 의미 (강사 강조)

--	--	--

구분	Supervised Learning (지도학습)	Unsupervised Learning (비지도학습)
데이터	라벨(정답, y)이 있는 데이터 $D=\{(X,y)\}$	라벨이 없는 데이터 $D=\{X\}$ 만 존재
목표	입력 $X \rightarrow$ 출력 y 로의 매핑함수 f 를 학습하여, 새로운 데이터의 정답을 예측	데이터 안에 숨겨진 구조·패턴을 스스로 발견 (정답 없이)
대표 문제	Classification(분류), Regression(회귀)	Clustering(군집화), 차원축소(PCA)
예시	이메일이 스팸인지 아닌지 분류, 집값 예측	고객을 유사한 그룹으로 나누기(세분화)
성능 평가	정답과 비교 가능 (Accuracy, MSE 등)	정답이 없어 평가가 상대적으로 어려움 (군집 내 응집도 등 간접 지표 사용)

- 의미: Supervised는 "정답이 있으니 정답에 가깝게 예측하는 함수를 찾는 것"이고, Unsupervised는 "정답 없이 데이터 자체의 내재된 패턴/구조를 찾는 것" — 이 본질적 차이를 서술할 수 있어야 함 (Short Answer 단골)
- Semi-supervised(준지도학습): 라벨 있는 데이터+없는 데이터 혼합 사용 $\rightarrow$ 지도학습의 확장 개념

시험 포인트

- "Sorting/shortest-path algorithm(Dijkstra)은 머신러닝인가?" $\rightarrow$ 아니오. 데이터로부터 학습하지 않으므로 ML이 아님 (Essay/T-F 단골)
- $AI \supset ML$, ML과 DM은 교집합 있지만 서로 포함관계 아님
- Supervised vs Unsupervised의 핵심 차이 = 라벨(정답)의 유무 — 이것이 목표·평가방법 모두를 결정함

1. Data Representation (데이터 표현)

영어	한국어
Scalar	스칼라 (숫자 하나)
Vector	벡터 (숫자 배열)
Matrix	행렬 (2차원 배열)
Tensor	텐서 (3차원 이상 배열)
Feature	특징/피처 (관측 가능한 개별 속성)
Sample / Instance	샘플/인스턴스 (분류·군집화 대상 하나)
Feature vector	특징 벡터
Feature extraction	특징 추출
Categorical variable	범주형 변수
Nominal variable	명목형 변수 (순서 없음, 예: 부서명)
Ordinal variable	순서형 변수 (순서 있음, 예: 평가등급)
Continuous variable	연속형 변수

- 표기법: 스칼라 = 소문자 이탤릭, 벡터 = 소문자 볼드, 행렬 = 대문자 볼드
- $X \in \mathbb{R}^{(m \times n)}$: 행(m)=샘플 수, 열(n)=특징 수 (선형대수 표기 관례 — 문제풀이에서 자주 나옴)
- y (반응변수, response variable): binary($\{0,1\}$ or $\{-1,1\}$), multi-label($\mathbb{Z}$), regression($\mathbb{R}$)

시험 포인트

- Nominal vs Ordinal 구분 (예시로 헷갈리게 낼 수 있음: "Excellent/Good/Poor" $\rightarrow$ Ordinal, "Computer/Biology/Physics" $\rightarrow$ Nominal)

2. Classification Problems (분류 문제) 개요

영어	한국어
Binary classification	이진 분류 (샘플 1개당 라벨 1개, 클래스 2개)
Multi-class classification	다중클래스 분류 (샘플 1개당 라벨 1개, 클래스 3개 $\uparrow$)
Multi-label classification	다중라벨 분류 (샘플 1개가 라벨 여러개 가질 수 있음)
Overfitting	과적합 (훈련데이터에 너무 딱 맞아 새 데이터에 일반화 실패)
Underfitting	과소적합 (모델이 너무 단순해서 패턴을 못 잡음)
Kernel trick	커널 트릭

시험 포인트

- Multi-class와 Multi-label 차이 (필수 암기 — Short Answer 단골)
- Overfitting = 훈련 성능은 높지만 테스트(새 데이터) 성능 낮음 / Underfitting = 둘 다 낮음

3. ☆☆ K-Nearest Neighbor, KNN (K-최근접 이웃) — 알고리즘/pseudocode 2문제 예상

- **Non-parametric (비모수적)**: 모델 파라미터 형태를 가정하지 않음
- **Lazy learning (게으른 학습)**: 학습 단계에서 "학습"하지 않고, 테스트 샘플이 들어올 때마다 K개 최근접 이웃을 찾음
- 분류 방식: **Majority vote (다수결 투표)** — K개 이웃 중 가장 많은 클래스로 결정

☆ KNN Pseudocode / 알고리즘 작동 방식 (강사 강조 — 반드시 설명 가능해야 함)

```
KNN(training_data, test_point, K):  
    1. training_data의 모든 점에 대해 test_point까지의 거리(distance)를 계산한다  
    2. 계산된 거리를 오름차순으로 정렬한다  
    3. 거리가 가장 가까운 상위 K개의 이웃(neighbor)을 선택한다  
    4. K개 이웃의 클래스 라벨을 확인하여, 다수결(majority vote)로  
       가장 많이 나온 클래스를 test_point의 예측 클래스로 결정한다
```

- **적용 가능한 문제 유형**: 기본적으로 Classification(분류)에 사용되지만, **Regression(회귀)에도 적용 가능** — 이웃 K개의 값을 평균 내어 예측값으로 사용
- KNN은 "학습"이라는 명시적 단계가 없고(Lazy learning), 새 데이터가 들어올 때마다 전체 훈련 데이터를 다시 훑어야 함 → 예측 시점에 계산비용이 큼

거리 척도 (Distance Measures) — Problem Solving 단골! (Essay/Workout 5문제 중 1문제 확정)

Euclidean Distance (유클리드 거리)

```
dist(a,b) = sqrt( ∑_{k=1}^p (a_k - b_k)^2 )
```

Minkowski Distance (민코프스키 거리) — 유클리드의 일반화

```
dist(a,b) = ( ∑ |a_k - b_k|^r )^(1/r)
```

- $r=1$ → **Manhattan/City block distance (맨해튼 거리)**, Hamming distance는 binary vector에서의 특수 예
- $r=2$ → **Euclidean distance**
- $r \rightarrow \infty$ → **Supremum distance (L^∞ norm)**, 두 벡터 성분 간 최대 차이값

Cosine Similarity (코사인 유사도)

```
cos(d1,d2) = (d1·d2) / (||d1|| ||d2||)
```

- 값 범위: 1(완전히 같음), 0(직교), -1(완전히 반대)
- 계산 예제 슬라이드에 있었음 ($d1 \cdot d2$ 내적 → 각 벡터 크기(norm) 나눠주기) — 직접 계산법 반드시 연습!

📌 Euclidean Distance 계산 연습문제 (시험에 확정적으로 1문제 나옴 — 반드시 손으로 풀어볼 것)

문제 예시: 다음 두 점 $a=(2, 4, 1)$ 과 $b=(5, 8, 2)$ 사이의 Euclidean distance를 구하시오.

풀이 과정

```
dist(a,b) = sqrt( (a1-b1)^2 + (a2-b2)^2 + (a3-b3)^2 )  
           = sqrt( (2-5)^2 + (4-8)^2 + (1-2)^2 )  
           = sqrt( (-3)^2 + (-4)^2 + (-1)^2 )  
           = sqrt( 9 + 16 + 1 )  
           = sqrt(26)  
           ≈ 5.099
```

계산 시 주의사항 (실수하기 쉬운 부분)

1. 각 차원(dimension)별로 **차이(difference)**를 먼저 구하고 제곱한다 (부호는 제곱하면 사라지므로 순서만 맞으면 됨)
2. 제공한 값들을 **전부 더한 후 마지막에 한 번만 제곱근(sqrt)**을 취한다 — 중간에 루트를 씌우는 실수 주의
3. 차원 수(p)가 몇 개든(2차원이든 3차원이든) 공식은 동일: **$dist = \sqrt{\sum (a_k - b_k)^2}$**
4. **계산기를 쓸 수 없으므로**, $\sqrt{26}$ 처럼 정확히 안 떨어지는 값은 소수점 근사값(≈ 5.10) 또는 $\sqrt{26}$ 형태로 그대로 답해도 무방한 경우가 많음 — 루트 안의 정수 계산까지는 반드시 정확히 맞출 것

Decision Boundary (결정 경계)

- **Voronoi diagram**: 각 점에 가장 가까운 영역을 나타내는 다이어그램. 두 클래스 경계는 두 점 사이의 수직이등분선
- **K 값의 효과**: K가 클수록 경계가 부드러워짐(smoother). $K=N$ 이면 항상 다수 클래스로 예측 (=상수 예측, 사실상 무의미)
- K가 작으면 → overfitting 위험 / K가 크면 → underfitting 위험 (일반화 경향)

Feature Scaling (특징 스케일링)

영어	한국어	공식
Standardization (Z-score)	표준화	$x_{norm} = (x - \mu) / \sigma$
Min-Max scaling	최소-최대 정규화	$x_{norm} = (x - x_{min}) / (x_{max} - x_{min})$

시험 포인트

- KNN Pros: 구현 단순/직관적, 유연한 결정 경계 / Cons: 불필요/상관된 특징에 민감, 고차원 데이터 어려움, 메모리·계산비용 큼
- Discussion 질문: "KNN을 회귀(regression)에 쓸 수 있는가?" → **Yes** (이웃들의 평균값 사용) — Short Answer/Essay 나올 수 있음
- Standardization이 왜 필요한가 → 스케일이 다른 특징이 거리 계산을 왜곡하기 때문

4. ☆☆ Evaluation & Preprocessing (평가 및 전처리) — Overfitting/CV "이유" 설명 중요

☆☆ 왜 Train-Test Split을 하는가 / 왜 K-fold CV를 하는가 (강사 강조, Short answer/Essay 단골)

- **왜 데이터를 훈련/테스트로 나누는가 (Why train-test split)?**
- 전체 데이터를 다 훈련에 쓰면 모델이 훈련 데이터에 과도하게 맞춰져(overfit) 100%에 가까운 성능을 보이지만, 이는 **훈련 데이터를 "외운"** 것일 뿐 새로운 데이터에 대한 일반화(generalization) 성능을 보장하지 않음
- 테스트 데이터는 모델이 한 번도 보지 않은 데이터로, **모델이 실제로 새로운 데이터에 얼마나 잘 작동하는지(일반화 성능)를 공정하게(cheating 없이) 평가하기** 위해 분리함
- **왜 K-fold Cross-Validation을 하는가 (Why K-fold CV)?**
- 단순 1회 Holdout 분할은 "운 나쁜 분할(unfortunate split)"이 발생하면 평가 결과가 왜곡될 수 있음(데이터가 한정적일수록 이 위험이 큼)
- K-fold CV는 데이터를 K개로 나누어 **모든 데이터가 훈련과 테스트에 골고루 한 번씩 사용**되도록 K번 반복 평가 → 특정 분할에 따른 우연성을 줄이고, **더 신뢰할 수 있는(robust) 성능 추정치**를 얻을 수 있음
- 특히 데이터가 적을 때 (제한된 데이터를 최대한 활용해야 할 때) 유용

Overfitting / Underfitting / Normalization / Optimization 요약

- **Overfitting(과적합):** 모델이 훈련 데이터의 노이즈까지 학습해 훈련 성능은 높지만 새 데이터(테스트)에서 성능이 낮음. 원인: 모델이 너무 복잡함, 데이터 부족. 해결: 정규화(Regularization), CV, 더 많은 데이터, Dropout 등
- **Underfitting(과소적합):** 모델이 너무 단순해서 훈련 데이터의 패턴조차 제대로 못 잡음. 훈련/테스트 성능 둘 다 낮음
- **Normalization(정규화, 스케일링):** 서로 다른 스케일의 feature들을 동일한 범위로 맞춰주는 전처리. Standardization(z-score), Min-Max scaling 등. KNN처럼 거리 기반 알고리즘에서 특히 중요
- **Optimization(최적화):** 손실함수(cost/loss function)를 최소화하는 파라미터를 찾는 과정. Gradient Descent가 대표적 (7번 섹션 참고)

Confusion Matrix (혼동 행렬)

	Predicted Yes	Predicted No
Actual Yes	TP (True Positive)	FN (False Negative)
Actual No	FP (False Positive)	TN (True Negative)

평가지표 (Evaluation Metrics) — 공식 암기 필수! Problem Solving 단골

영어	한국어	공식
Sensitivity / True Positive Rate (TPR), Recall	민감도/재현율	$TP / (TP + FN)$
Specificity / True Negative Rate (TNR)	특이도	$TN / (FP + TN)$
Precision / Positive Predictive Value (PPV)	정밀도	$TP / (TP + FP)$
Negative Predictive Value (NPV)	음성예측도	$TN / (TN + FN)$
Accuracy	정확도	$(TP + TN) / (TP + FP + TN + FN)$
False Positive Rate (FPR)	위양성률	$FP / (FP + TN) = 1 - \text{Specificity}$

- **Accuracy의 한계 (Limitation of Accuracy):** 클래스 불균형(imbalanced data)일 때 정확도가 왜곡됨. 예: 9990:10 데이터에서 전부 다수클래스로 찍어도 정확도 99.9% → 무의미한 모델. **자주 나오는 서술형/T-F 문제!**

ROC Curve (ROC 곡선)

- x축 = FPR, y축 = TPR
- (0,0): 전부 negative로 예측 / (1,1): 전부 positive로 예측 / **(0,1): 완벽한(perfect) 분류기** (주의: 슬라이드 원문은 (1,0)이 "perfect"라 적혀 있으나 일반적으로 TPR=1, FPR=0인 좌상단이 이상적 지점 — 강의자료 표기 그대로 암기 권장: TPR축이 y, FPR축이 x이므로 좌상단 = TPR 1, FPR 0)

- 대각선(diagonal line) = random guessing (50%)
- **AUC (Area Under Curve)**: 모델의 평균적 성능 측정, 다른 모델과 비교하기 좋음

Cross-Validation (교차검증) 방법들

영어	한국어	핵심
Holdout	홀드아웃	2/3 훈련, 1/3 테스트로 1회 분할
Random subsampling	랜덤 서브샘플링	복원없이 랜덤 샘플링 반복
K-Fold Cross-validation	K-겹 교차검증	K개로 등분, K-1개 훈련+1개 테스트, K번 반복 후 평균 오차 $E = (1/K) \sum E_i$
Leave-one-out (LOOCV)	리브원아웃	N-1개 훈련, 1개 테스트 (N=K-fold의 특수케이스)
Stratified CV	층화 교차검증	각 fold에서 클래스 비율 유지 (불균형 데이터에 중요)
Bootstrapping	부트스트래핑	복원추출(sampling with replacement)

- **Fold 수 트레이드오프**: fold 많음 → bias 작음(정확), but 계산비용 큼 / fold 적음 → 계산 저렴, variance 작음, bias 큼
- **5-fold 또는 10-fold CV가 일반적으로 가장 흔한 선택**
- Oversampling(오버샘플링): 소수 클래스 증폭 / Undersampling(언더샘플링): 다수 클래스 축소
- **CV + 정규화 규칙(중요, Essay 단골)**: 정규화(z-score 등)는 **훈련 데이터로만 평균·표준편차를 구하고**, 그 값을 검증/테스트 데이터에도 동일하게 적용해야 함. 그렇지 않으면 테스트 데이터가 훈련 정보를 "누수(leak)"하게 됨 → data leakage/cheating

시험 포인트

- Test data는 절대 모델 생성(훈련)에 사용하면 안 됨 → "cheating"
- Precision vs Recall 실제 계산 문제 나올 가능성 높음 (Confusion matrix 주고 계산 → Problem Solving)

5. ☆☆ Correlation & Linear Regression (상관관계 & 선형회귀) — 최적 파라미터 찾는 법 & Multicollinearity

- **Correlation (상관계수, r)**: 두 변수의 선형 연관성. 범위 -1~+1. r=0 → 상관관계 없음. **Pearson correlation coefficient**가 가장 흔히 쓰임

Linear Regression Model (선형회귀 모델)

$$y = b_0 + bX + \epsilon \quad (\epsilon \sim N(\mu, \sigma^2) \text{ 가정})$$

- Simple linear regression(단순선형회귀): 독립변수 1개
- Multiple linear regression(다중선형회귀): 독립변수 2개 이상
- 행렬형태: **$y = Xb + e$** , X는 절편(intercept)을 위해 1로 채운 열 포함

OLS (Ordinary Least Squares, 최소제곱법) — 공식 유도 암기!

- 목적함수: 오차제곱합(SSE) 최소화

$$\min J(b) = \sum (y_i - x_i \cdot b)^2$$

- 닫힌 형태(closed-form) 해:

$$b = (X'X)^{-1} X'y$$

- 유도과정: $J = e'e = y'y - 2b'X'y + b'X'Xb \rightarrow \partial J / \partial b = -2X'y + 2X'Xb = 0 \rightarrow X'Xb = X'y$

범주형 변수 처리

- Ordinal → 연속값으로 인코딩 (예: Excellent=5...Very poor=1)
- Nominal → **Dummy variable (더미 변수)**, one-hot encoding 방식

선형회귀를 분류에 사용 (Linear regression for classification)

- y를 {0,1} 또는 {-1,1}로 인코딩 후 OLS 적용, 임계값(threshold, λ)로 클래스 결정
- **한계**: Linear model은 분류에 최적화된 게 아님 → Logistic Regression으로 이어짐 (다음 섹션과 연결되는 논리 흐름, Essay에서 "왜 로지스틱 회귀가 필요한가"를 수 있음)

Assumptions in Linear Regression (선형회귀 가정)

1. 독립변수와 종속변수 간 선형성 (Linearity)
2. 오차항이 평균 0, 등분산의 정규분포를 따름

3. 독립변수들끼리 서로 독립 (violate 시 **Multicollinearity**, **다중공선성** 문제 발생 → 변수 제거 필요)

시험 포인트

- OLS가 왜 절대값이 아니라 제곱을 쓰는가? → 미분가능/불록함수라 최적화 쉬움, 큰 오차에 더 페널티
- Statistics 관점(설명력/p-value 중요) vs CS 관점(예측 정확도 중요) 차이 — Essay 단골

6. ☆☆ Regularization — Ridge Regression (릿지 회귀, 정규화) — Penalty term의 의미

☆☆ Multicollinearity(다중공선성) 문제란? (강사 강조)

- 독립변수(feature)들끼리 서로 강하게 상관되어 있는 경우 발생
- 문제점:** OLS로 계수(b)를 추정할 때 행렬 $X'X$ 가 특이행렬(singular)에 가까워져 **역행렬이 불안정하거나 존재하지 않음** → 계수 추정값이 극단적으로 크거나 부호가 이상하게 나오는 등 신뢰할 수 없는 결과 발생 (강의자료 예시: 실제로는 (1,1)에 가까운 계수인데 추정은 (40,-38)처럼 엉뚱하게 나옴)
- 해결책 = Ridge Regression** (penalty term 도입)
- 동기(Motivation):** 독립변수들이 서로 높은 상관관계(collinearity)를 가지면 계수 추정이 불안정해짐
- Ridge Regression 목적함수:**

$$\text{Minimize } \sum (y_i - Xb)^2 + \lambda ||b||^2 \quad (\text{L2-norm penalty})$$

- ☆ **Penalty term(페널티 항)의 의미:** $\lambda ||b||^2$ 항은 계수(b)가 너무 커지는 것을 억제하는 "벌점". 계수가 클수록 목적함수 값이 커지므로, 모델은 데이터에 대한 적합도(fit)와 계수 크기 억제 사이에서 균형을 찾게 됨
- 계수를 작게(0에 가깝게) **수축(shrink)**시켜서 다중공선성으로 인한 계수의 불안정성을 줄이고, 모델의 분산(variance)을 낮춰 일반화 성능을 향상시킴
- λ (람다)는 이 벌점의 세기를 조절하는 **정규화 파라미터(regularization parameter)** — λ 가 클수록 계수를 더 강하게 억제
- Lagrange Multiplier(라그랑주 승수):** 제약조건이 있는 최적화를 위한 기법. 제약 $\min \sum f(x) \text{ s.t. } g(x) \leq C \leftrightarrow \min \sum f(x) + \lambda g(x)$
- 닫힌형태 해:

$$b = (X'X + \lambda I)^{-1} X'y$$

- $X'X + \lambda I$ 는 항상 역행렬이 존재**(invertible) → 항상 유일한 해를 가짐 (OLS와 달리 다중공선성 문제에도 안정적)
- $\lambda \rightarrow 0$ 이면 Ridge 해 → OLS 해로 수렴 / $\lambda \rightarrow \infty$ 이면 계수들이 0으로 수축(shrink)
- Bias-Variance tradeoff:** bias는 증가하지만 variance는 감소 (일반화 성능 향상 목적)
- P-norm: $p=1$ (Manhattan/L1 norm), $p=2$ (Euclidean/L2 norm), $p=\infty$ (maximum norm)

시험 포인트

- Ridge는 왜 항상 해가 존재하는가? ($X'X + \lambda I$ 가 항상 invertible) — 자주 나오는 개념 질문

7. Optimization & Gradient Descent (최적화 & 경사하강법)

영어	한국어
Global optimization	전역 최적화 (모든 가능한 해 중 최적)
Local optimization	지역 최적화 (근방 후보해 중 최적)
Convex set	볼록 집합
Convex function	볼록 함수
Gradient	그래디언트(기울기 벡터)
Learning rate / step size	학습률/스텝사이즈 (α)

- Convex set 정의:** 집합 내 임의의 두 점을 이은 선분이 모두 그 집합 안에 있으면 convex
- Convex function 정의:** 그래프 위 두 점을 잇는 직선이 항상 함수 그래프 위(또는 같음)에 있으면 convex
- Gradient (∇f):** 각 변수에 대한 편미분 벡터. **가장 가파르게 증가하는 방향(steepest ascent)**을 가리킴 → 그래서 경사하강법은 **음의 방향**으로 이동

Gradient Descent 알고리즘 (경사하강법)

$$\theta := \theta - \alpha * \partial J / \partial \theta \quad (\text{모든 파라미터에 대해 동시 업데이트})$$

- α (학습률) 너무 크면 → 최적점을 지나침(overshoot/jump), 발산 가능
- α 너무 작으면 → 수렴이 느림

- **Convex objective** → **global minimum 수렴 보장** / Non-convex → local minimum에 갇힐 위험 (다른 초기값으로 완화 가능)

기타 수치 최적화 기법 (이름만 알아두기)

Steepest descent, Newton Method, Gauss-Newton, Levenberg-Marquardt, Line Search, Trust-Region

시험 포인트

- " ∇J 가 가리키는 방향"과 "우리가 실제 이동하는 방향(반대)" 구분 — T/F 함정 단골
- Convex 함수여야 global minimum 보장 (non-convex면 아님)

8. ☆ Logistic Regression & Softmax (로지스틱 회귀 & 소프트맥스) — 약 1문제 출제

왜 선형회귀로 분류하면 안 되는가

- 이진 반응변수($y=\{0,1\}$)에 선형회귀를 적용하면 예측값이 $[0,1]$ 범위를 벗어날 수 있음 → 확률 해석 불가

Sigmoid function (시그모이드 함수)를 통한 확률적 접근

$$P(Y=1|X) = \frac{1}{1 + \exp(-Xb)} = \frac{\exp(Xb)}{1 + \exp(Xb)}$$
$$P(Y=0|X) = \frac{1}{1 + \exp(Xb)}$$

Logit (로짓) / Log-odds

$$\log\left(\frac{p}{1-p}\right) = Xb$$

- $p/(1-p)$ = **Odds ratio (오즈비)**, 범위 $[0,\infty)$
- $\log(\text{odds})$ = **Logit**, 범위 $(-\infty,\infty)$ → 그래서 선형모델에 적합

MLE (Maximum Likelihood Estimation, 최대우도추정)

- 우도함수: 관측데이터가 나올 확률을 파라미터의 함수로 봄

$$L(b;X) = \prod p(x_i)^{y_i} (1-p(x_i))^{(1-y_i)}$$

- **Log-likelihood (로그우도)**로 변환하여 곱을 합으로 (계산 편의)

$$\sum [y_i \cdot \log(p(x_i)) + (1-y_i) \cdot \log(1-p(x_i))]$$

- **Negative log-likelihood (음의 로그우도, NLL)**를 최소화하는 문제로 변환 → Gradient Descent로 해결(단힌형태 해 없음, OLS와 차이!)

Softmax Classifier (소프트맥스 분류기)

- 로지스틱 회귀의 **다중클래스 일반화** (multi-label classification)
- 딥러닝에서 널리 사용

$$p(y=k|X) = \frac{\exp(y_k)}{\sum_j \exp(y_j)}$$

- 로지스틱은 정규화 안 된 점수를 시그모이드로 $[0,1]$ 에 매핑, 소프트맥스는 여러 클래스 점수를 정규화(합=1)

시험 포인트

- 로지스틱 회귀는 **단힌 형태 해가 없다** (선형/릿지회귀와 대비, MLE→gradient descent로 근사) — 자주 나오는 T/F
- Odds vs Logit vs Probability 세 개념 구분, 계산 문제 나올 수 있음 (Problem Solving)

9. ☆☆☆ Neural Networks (신경망) & Backpropagation — 최우선 출제 범위

강사가 리뷰세션에서 가장 강조한 파트. Essay/Short Answer에서 비중 있게 다뤄질 가능성이 매우 높으니 이 섹션은 반드시 "설명할 수 있는 수준"까지 익힐 것.

9-1. 왜 단일 퍼셉트론으로는 부족한가 (Why single Perceptron is not enough)

- 단일 퍼셉트론(Perceptron)은 **선형적으로 분리 가능한(linearly separable)** 데이터만 분류할 수 있음
- 결정경계가 직선(초평면) 하나뿐이라, 비선형 패턴을 표현할 수 없음

- **대표 반례 = XOR 문제**: XOR은 하나의 직선으로 절대 분리할 수 없는 대표적인 비선형 분리 문제 → 단일 퍼셉트론은 XOR을 학습할 수 없음
- 이 한계가 1950년대에 발견되면서, 이후 1970년대까지 신경망 연구가 침체됨(AI Winter의 한 원인)

9-2. 왜 다층 퍼셉트론(MLP)이 필요한가 (Why Multi-layer Perceptron)

- **Hidden layer(은닉층)을 추가**하면, 여러 개의 선형 결정경계를 조합해서 **비선형 결정경계**를 만들 수 있음
- MLP는 XOR과 같이 선형분리 불가능한 문제도 해결 가능 (레이어를 쌓아 비선형 함수 $f(x)=f^3(f^2(f^1(x)))$)를 구성)
- **Universal Approximator Theorem**: hidden layer가 1개만 있어도 이론적으로는 임의의 함수를 근사할 수 있음
- 그런데도 **왜 더 깊게(deep) 쌓는가?** → 얇은 망(shallow net)은 같은 표현력을 얻으려면 지수적으로 더 넓은 width가 필요하고, overfitting 경향도 더 큼. Deep network가 계층적으로 특징을 학습하여 더 효율적

9-3. 신경망 구조 (Architecture) 설계

영어	한국어
Input layer	입력층
Hidden layer	은닉층
Output layer	출력층
Depth	깊이 (layer 수)
Width	너비 (한 layer의 unit 수)
Feedforward Network / MLP	순전파 신경망/다층퍼셉트론

- Weighted input: $z_j^l = \sum_k w_{jk}^l \cdot a_k^{(l-1)} + b_j^l$
- Activation: $a_j^l = f(z_j^l)$

★ 입력층/출력층 노드 수는 왜 그렇게 정하는가 (자주 나오는 질문)

- **입력층(Input layer) 노드 수 = 데이터의 feature(특징) 개수**

- 예: MNIST 손글씨 데이터가 28×28 픽셀 이미지라면 입력층 노드 수 = 28×28 = 784개

- **출력층(Output layer) 노드 수는 문제 유형에 따라 결정**

- Binary classification(이진분류): 출력 노드 1개 (sigmoid로 0~1 확률 출력) 또는 2개(softmax)

- Multi-class classification(다중클래스분류): 클래스 개수만큼 (예: MNIST는 0~9 숫자 → 출력노드 10개, softmax 사용)

- Regression(회귀): 출력노드 1개 (예측하려는 연속값 1개, activation 없이 또는 linear)

- 즉, **네트워크 구조는 "데이터가 몇 개의 특징을 가지는가"와 "무엇을 예측해야 하는가"에 의해 결정됨** — Essay에서 "MNIST를 위한 신경망을 설계하라"는 식의 질문(강의 Discussion 슬라이드에 실제로 있었음)에 이 논리로 답할 것

9-4. Activation Functions (활성화 함수)와 선택 이유

이름	공식	주로 쓰이는 곳
Sigmoid	$1/(1+e^{-z})$	은닉층(과거), 이진분류 출력층 — 단점: 기울기 소실 유발
Softmax	$e^{z_j} / \sum_k e^{z_k}$	다중클래스 분류의 출력층 (확률 합=1로 정규화)
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	은닉층 (sigmoid보다 중심이 0이라 학습에 유리)
ReLU (Rectified Linear Unit)	$\max(0, z)$	현재 은닉층 표준 — 계산 단순, vanishing gradient 완화

- 활성화 함수가 없다면(즉 선형함수만 쌓으면) 아무리 layer를 쌓아도 결국 하나의 선형 변환과 동일 → **비선형 활성화 함수가 신경망의 표현력의 핵심**

Cost Functions (비용함수)

- Quadratic cost (제곱오차): $C = (1/2n) \sum ||y - a^L||^2$
- Binary Cross-Entropy cost
- Negative Log-likelihood cost: $C = -\log(a_y^L)$
- 조건: (1) 개별 샘플 비용의 평균이어야 함 (2) 신경망 출력의 함수여야 함 (backprop 가능하려면)

9-5. Forward Pass (순전파) — 단계별 과정 (Essay 단골, 순서대로 설명 가능해야 함)

1. **입력(Input)**: 입력 벡터 x를 입력층의 활성화 a^1 로 설정
2. **가중합 계산(Weighted sum)**: 각 layer에서 $z^l = w^l \cdot a^{(l-1)} + b^l$ 계산 (이전 layer 출력 × 가중치 + 편향)
3. **활성화(Activation)**: $a^l = f(z^l)$ — 활성화 함수를 통과시켜 비선형성 부여
4. 이 과정을 입력층 → 은닉층(들) → 출력층까지 **순서대로(forward)** 반복
5. **출력(Output)**: 출력층의 활성화 a^L 이 최종 예측값 (분류면 확률, 회귀면 예측값)
6. 예측값 a^L 과 실제 정답 y를 비교하여 **Cost function(비용함수) C** 계산

9-6. Backpropagation (역전파) — 원리와 5단계

- **핵심 원리 = Chain Rule (연쇄법칙)**: $dz/dx = dz/dy \cdot dy/dx$ — 출력층의 오차를 각 layer의 가중치에 대한 gradient로 "거슬러 전파"시킴

- 동적계획법(dynamic programming)으로 반복 서브식(subexpression) 재계산 방지 → 속도 vs 메모리 트레이드오프
- 오차 $\delta_j^l = \partial C / \partial z_j^l$ 정의
- 출력층 오차: $\delta^L = \partial C / \partial a^L \odot \sigma'(z^L)$
- 가중치 업데이트: $\partial C / \partial w_{jk}^l = a_k^{l-1} * \delta_j^l$
- 편향 업데이트: $\partial C / \partial b_j^l = \delta_j^l$

Backpropagation Algorithm 순서 (5단계, 반드시 암기!)

- Input:** x를 입력층 활성화 a^1 로 설정
- Feedforward (순전파):** 각 layer $l=2, \dots, L$ 에서 z^l, a^l 계산
- 출력층 오차 δ^L 계산:** 비용함수를 출력에 대해 미분
- Backpropagate (역전파):** $l=L-1, L-2, \dots, 2$ 순서로 역방향으로 δ^l 계산
- 가중치/편향 업데이트:** $\partial C / \partial w, \partial C / \partial b$ 를 이용해 gradient descent로 파라미터 갱신

왜 backprop이 필요한가 (Essay에서 나올 수 있는 "설명" 질문): 신경망은 파라미터(가중치·편향)가 매우 많기 때문에, 각 파라미터에 대한 gradient를 하나씩 직접 계산하면 매우 비효율적. Backprop은 chain rule + dynamic programming으로 **모든 layer의 gradient를 한 번의 역방향 계산으로 효율적으로 구하는 알고리즘**.

Vanishing Gradient Problem (기울기 소실 문제)

- 원인: 깊은 layer를 거치며 기울기가 지수적으로 작아짐 (특히 sigmoid 사용 시, chain rule로 작은 값들이 계속 곱해지기 때문)
- 해결책: **ReLU 등 다른 활성화함수 사용, Batch Normalization(배치 정규화)**

Gradient Descent 변형들

영어	한국어	설명
Stochastic Gradient Descent (SGD)	확률적 경사하강법	샘플 1개마다 업데이트
Batch Gradient Descent	배치 경사하강법	전체 데이터 다 본 후(1 epoch) 업데이트
Mini-batch Gradient Descent	미니배치 경사하강법	작은 배치 단위로 업데이트, 딥러닝에서 가장 흔함(SGD와 배치의 균형)

Overfitting 방지 (신경망)

- Regularization (정규화), Dropout/DropConnect

시험 포인트 (☆☆☆ 최우선)

- "단일 퍼셉트론이 왜 부족한가?" → 선형보리만 가능, XOR 예시로 설명 (Short answer/Essay 최다빈출)
- "왜 MLP가 필요한가?" → hidden layer로 비선형 결정경계 생성 가능
- Forward pass 단계를 순서대로 설명 (Essay)
- Backprop 5단계 순서 서술 + "왜 필요한가"까지 설명 (Essay 단골)
- 입력층/출력층 노드 수를 어떻게 정하는지 (feature 개수 / 클래스 개수) — Short answer 단골
- Vanishing gradient 원인과 해결책 (Short answer 단골)
- SGD vs Batch vs Mini-batch 차이 명확히 구분
- 활성화 함수별 특징과 어디에 쓰이는지 구분

10. Convolutional Neural Networks, CNN (합성곱 신경망)

영어	한국어
Convolutional layer	합성곱층
Filter / Kernel	필터/커널
Stride	스트라이드 (필터 이동 간격)
Padding	패딩 (입력 가장자리에 0 채우기)
Pooling layer	풀링층
Max pooling	맥스풀링 (가장 흔함)
Feature map	특징 맵
ReLU	렐루 (음수 픽셀값을 0으로)

- Hyperparameters: Stride(필터 이동량), Padding size(경계 zero-padding 크기)
- Pooling 목적: 공간 크기(파라미터 수) 점진적 축소
- CNN 대표 모델 (연도/특징 암기)**
- LeNet:** 최초 성공적 CNN (Yann LeCun, 1990s)

- **AlexNet**: CNN 대중화시킨 첫 연구 (Alex Krizhevsky, 2012)
- **GoogLeNet**: ILSVRC 2014 winner
- **VGGNet**: ILSVRC 2014 runner-up
- **ResNet**: Residual Network, ILSVRC 2015 winner (Kaiming He)

시험 포인트

- 모델 이름-연도-특징 매칭 (T/F 단골: "AlexNet은 2012년 최초의 CNN이다" → **False**, 최초 CNN은 LeNet, AlexNet은 대중화시킨 첫 사례)
- Convolution vs Pooling layer 역할 차이

11. ☆☆ Support Vector Machine, SVM (서포트 벡터 머신) — 개념 위주 2문제 (계산보다 "이해")

강사 공지: SVM은 수학적 계산보다 **개념(concept)** 위주로 약 2문제 출제됩니다. 아래 두 가지를 말로 설명할 수 있으면 충분합니다:

1. 왜 **margin**을 최대화하는가 (Why maximize the margin)?
2. **선형 분리(Linear separation)** 판별자(discriminant)란 무엇인가?

☆ 왜 Margin을 최대화하는가 (Why maximize the margin) — 개념 설명

- SVM은 두 클래스를 나누는 hyperplane(초평면) 중에서, **결정 경계와 가장 가까운 데이터 포인트(어려운 점, difficult points)까지의 거리(margin)를 최대화**하는 hyperplane을 선택함
- **직관**: 결정 경계 근처에 아무 데이터도 없다면, 새로운 데이터에 대한 분류 결정이 훨씬 더 "확실(certain)"해짐. 반대로 margin이 좁으면 경계 근처의 약간의 노이즈나 새로운 데이터만으로도 분류가 쉽게 뒤바뀔 수 있음
- 즉, Margin을 최대화하는 것은 **분류기의 일반화 성능(generalization)을 높이고, 새로운 데이터에 대해 더 강건(robust)한 결정 경계를 만들기 위함**
- Margin을 정의하는(=결정 경계에 가장 가까운) 데이터 포인트들이 바로 **Support Vectors(서포트 벡터)**이며, 이 점들만으로 결정함수가 완전히 결정됨(나머지 데이터는 결정경계에 영향 없음)

☆ Linear Separator / Discriminant Function (선형 분리 판별자) 개념

- 이진분류(binary classification)는 **feature space(특징 공간)에서 두 클래스를 분리하는 문제로 볼 수 있음**
- 판별함수(discriminant function): $f(x) = \text{sign}(w^T x + b)$
- w : 가중치 벡터(weight vector) — hyperplane에 수직인 법선벡터(normal vector)
- b : 편향(bias)
- $w^T x + b = 0$ 이 결정 경계(decision boundary, hyperplane) 자체를 정의
- $w^T x + b > 0$ 이면 한 클래스, < 0 이면 다른 클래스로 판별
- 여러 개의 hyperplane이 데이터를 분리할 수 있는 경우(가능한 a,b,c 여러 개), **SVM은 그 중 margin이 최대인 최적의 hyperplane 하나를 찾음**

기본 개념

- **SVM = margin(마진)을 최대화하는 hyperplane(초평면)을 찾는 분류기**
- **Support vectors (서포트 벡터)**: 결정 경계에 가장 가까운, 결정함수를 정의하는 데이터 포인트들
- Solving SVM = **Quadratic programming (2차계획법)** 문제

(참고) Margin 공식 — 계산 문제로는 안 나올 가능성이 높지만, 개념 이해를 돕기 위해 남겨둠

- 기하학적 마진: $r = y(w^T x + b) / ||w||$
- Functional margin(함수적 마진): $y_i(w^T x_i + b)$
- 선형분리 가능 케이스 제약조건:

$$\begin{aligned} w^T x_i + b &\geq 1 & \text{if } y_i = 1 \\ w^T x_i + b &\leq -1 & \text{if } y_i = -1 \end{aligned}$$

- 마진 폭(margin width): $\rho = 2 / ||w||$

최적화 문제 (Primal → Dual)

- Primal: $\min \Phi(w) = \frac{1}{2} w^T w$ s.t. $y_i(w^T x_i + b) \geq 1$
- Dual (Lagrange multiplier α 사용):

$$\begin{aligned} \text{maximize } Q(\alpha) &= \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j x_i^T x_j \\ \text{subject to: } &\sum \alpha_i y_i = 0, \quad \alpha_i \geq 0 \end{aligned}$$

- 해: $w = \sum \alpha_i y_i x_i$, $b = y_k - w^T x_k$ (support vector 기준)
- $\alpha_i \neq 0$ 인 **xi만 support vector** — 나머지 데이터는 결정경계에 영향 없음

Soft Margin Classification (소프트 마진, 데이터가 선형분리 불가능할 때)

- Slack variable(여유변수) ξ 도입: 오분류 허용

$$\begin{aligned} \phi(w) &= \frac{1}{2}w^T w + C \sum \xi_i && \text{최소화} \\ \text{s.t. } y_i(w^T x_i + b) &\geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

- 파라미터 C:** 과적합 통제용 정규화 항 — C 클수록 오분류에 더 엄격(margin 좁아짐), C 작을수록 오분류 허용 큼(margin 넓어짐)

Kernel Trick (커널 트릭)

- 선형분리 불가능한 데이터를 고차원 공간으로 매핑($\Phi: x \rightarrow \phi(x)$)해서 분리 가능하게 만들
- $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ — 커널함수는 고차원 공간에서의 내적과 동일한 값을 저차원 계산으로 구함(계산량 절감)
- 대표 커널: Linear, **Polynomial:** $K(x,z)=(1+x^Tz)^d$, **RBF(Radial Basis Function, 무한차원)**

시험 포인트

- Margin 계산 (Worked example처럼 w, b 구하는 문제) — **Problem Solving 최유력 출제 파트**
- Soft margin의 파라미터 C 역할 (Short answer/Essay 단골)
- Support vector의 정의 ($\alpha \neq 0$ 인 데이터포인트)

시험 직전 최종 체크리스트 (강사 리뷰세션 공지 반영)

✦ 확정된 계산 문제 — 반드시 손으로 풀 것

- Euclidean Distance 계산 (확정, Essay/Workout 5문제 중 1문제)** → 3번 섹션 "계산 연습문제" 참고, 최소 3세트 이상 직접 풀어볼 것

☆☆☆ 최우선 — Neural Networks & Backpropagation (비중 가장 큼)

- 단일 퍼셉트론의 한계 (선형분리만 가능, XOR로 설명)
- MLP가 필요한 이유 (hidden layer로 비선형 표현)
- Forward pass 단계별 과정 순서대로 설명
- Backpropagation 5단계 + "왜 필요한가"
- 입력층/출력층 노드 수를 정하는 기준 (feature 개수 / 클래스 개수)
- 활성화 함수 종류와 특징 (Sigmoid/Tanh/ReLU/Softmax)
- Vanishing gradient 원인과 해결책

☆☆ KNN (약 2문제, 알고리즘/pseudocode 중심)

- KNN pseudocode 순서 설명 가능해야 함
- KNN을 적용 가능한 문제 유형 (분류뿐 아니라 회귀에도 사용 가능)
- Euclidean/Manhattan/Minkowski distance 계산 (공식 암기 + 직접 계산)

☆☆ SVM (약 2문제, 개념 위주 — 계산 부담 적음)

- 왜 margin을 최대화하는가 (일반화 성능, robust한 결정경계)
- Linear separator/discriminant function 개념 ($w^T x + b = 0$)
- Support vector의 정의

☆☆ Supervised vs Unsupervised Learning

- 라벨(정답) 유무 차이, 목표와 평가방법의 차이를 설명할 수 있을 것

☆☆ Data Evaluation & Preprocessing

- Train-test split을 하는 이유 / K-fold CV를 하는 이유 (일반화 성능의 공정한 평가)
- Overfitting vs Underfitting 구분
- Confusion Matrix → Precision/Recall/Accuracy/Sensitivity/Specificity 계산 연습

☆☆ Linear & Ridge Regression

- OLS 계수 $b = (X^T X)^{-1} X^T y$ — 최적 파라미터를 찾는 방법(유도 논리) 이해
- Multicollinearity(다중공선성) 문제란 무엇이고 왜 발생하는가
- Ridge Regression 공식 $b = (X^T X + \lambda I)^{-1} X^T y$ 와 **Penalty term($\lambda \|b\|^2$)의 의미** (계수 수축, 분산 감소)

☆ Logistic Regression (약 1문제)

- Sigmoid/Logit/Odds 상호 변환 계산

- Linear regression으로 분류하면 안 되는 이유 → Logistic Regression이 필요한 이유
- 닫힌 형태 해가 없어 Gradient Descent로 근사한다는 점

Essay 예상 주제 (개념 비교·설명형, 우선순위 순)

- **Backpropagation 원리와 5단계 + Forward pass 과정** (최우선)
- 단일 퍼셉트론 vs MLP (왜 MLP가 필요한가)
- SVM margin maximization 아이디어, soft margin의 필요성
- Supervised vs Unsupervised Learning 차이
- Train-test split과 K-fold CV를 하는 이유
- OLS vs Ridge Regression (Multicollinearity 해결과 penalty term의 효과)
- AI vs Data Mining vs Machine Learning 차이
- Overfitting vs Underfitting
- Logistic Regression이 필요한 이유 (Linear regression의 한계)
- SGD vs Batch vs Mini-batch Gradient Descent

True/False 단골 함정

- "단일 퍼셉트론으로 XOR을 분류할 수 있다" (**False**, 선형분리 불가능하므로 MLP 필요)
- "입력층 노드 수는 임의로 정해도 된다" (**False**, feature 개수와 같아야 함)
- "OLS/Ridge에는 닫힌 형태 해가 있다" (True) vs "Logistic Regression에도 닫힌 형태 해가 있다" (**False**, MLE→gradient descent)
- "K가 클수록 KNN decision boundary가 거칠어진다" (**False**, 더 부드러워짐)
- "KNN은 분류에만 사용할 수 있다" (**False**, 회귀에도 적용 가능)
- "정확도(accuracy)는 불균형 데이터에서도 좋은 지표다" (**False**)
- "테스트 데이터로 정규화 파라미터(평균/표준편차)를 계산해도 된다" (**False**, 훈련데이터로만)
- " ∇f 는 가장 가파르게 감소하는 방향을 가리킨다" (**False**, 증가 방향. 그래서 반대로 이동)
- "AlexNet이 최초의 CNN이다" (**False**, LeNet이 최초, AlexNet은 대중화)
- "Ridge Regression의 $X'X + \lambda I$ 는 항상 역행렬을 갖는다" (**True**)
- "Multicollinearity가 있어도 OLS 계수 추정에는 문제가 없다" (**False**, 불안정한 추정)

이 노트는 강의안 358페이지 전체를 기반으로 작성되었습니다. 시험 전 이 문서를 훑고, 위 "최종 체크리스트"의 계산 문제 유형을 손으로 직접 풀어보는 것을 추천합니다.